

Change is on the Horizon



Chaos: Change that is unpredictable

Learning objectives

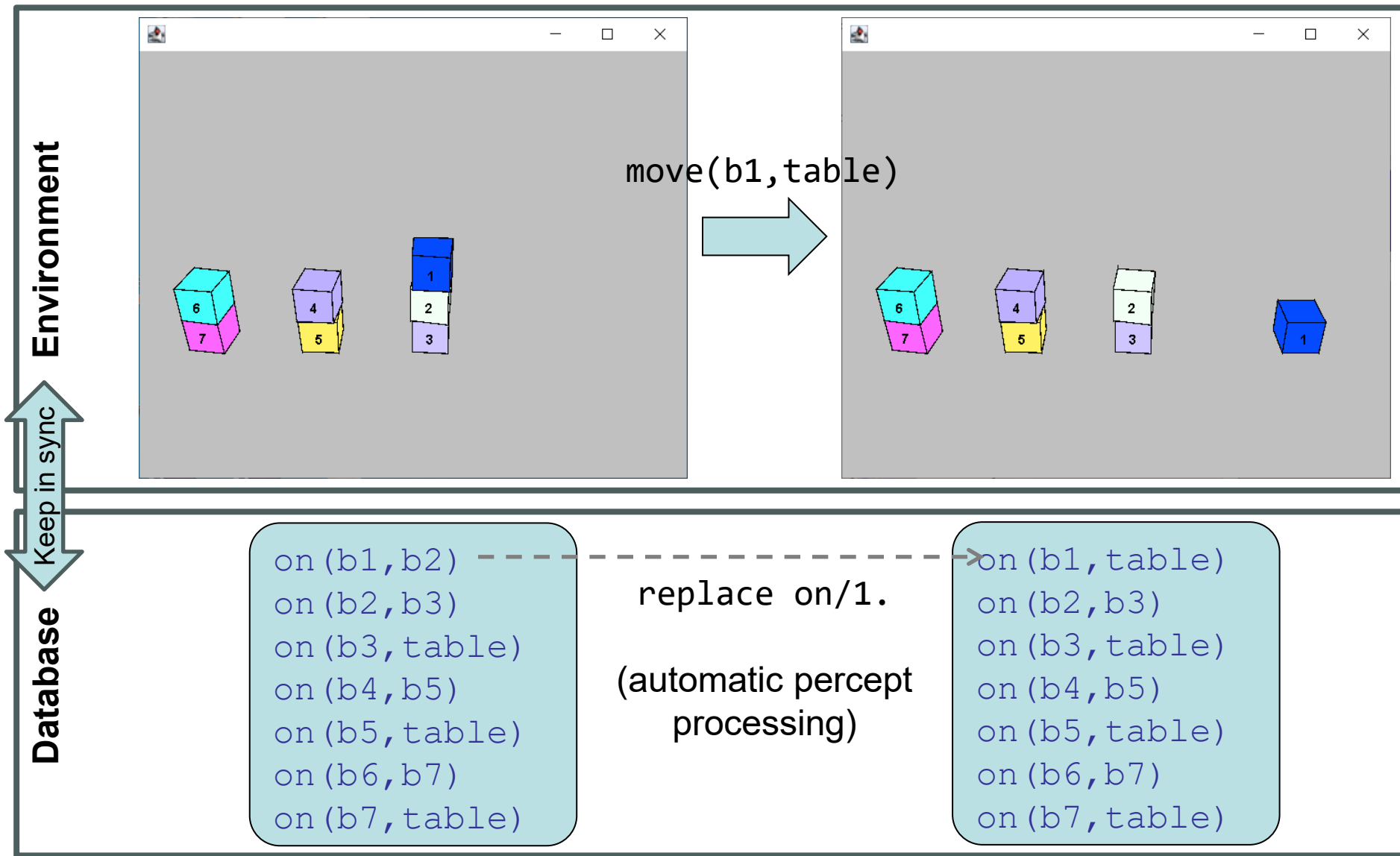
Understand how to program a MARBEL agent:

- Properties of environments.
- Representing and reasoning about environments.
- Durative actions.
- Prioritizing actions by means of rule order.
- Debugging and refining an agent program.
- Basic understanding of modules.

Agenda

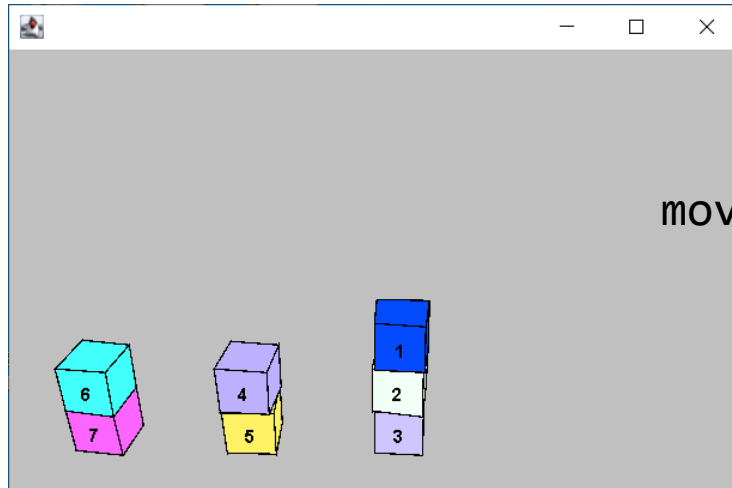
- Blocks World recap. Blocks World vs Tower World.
- Durative actions.
- Recap: Tower World environment properties.
- Representing & reasoning about the Tower World.
- Common sense design of agent code (strategy).
- Action rule evaluation: order.
- Putting it all together & Solving the Tower World.
- Some basic debugging skills (along the way).

Recap: Blocks World

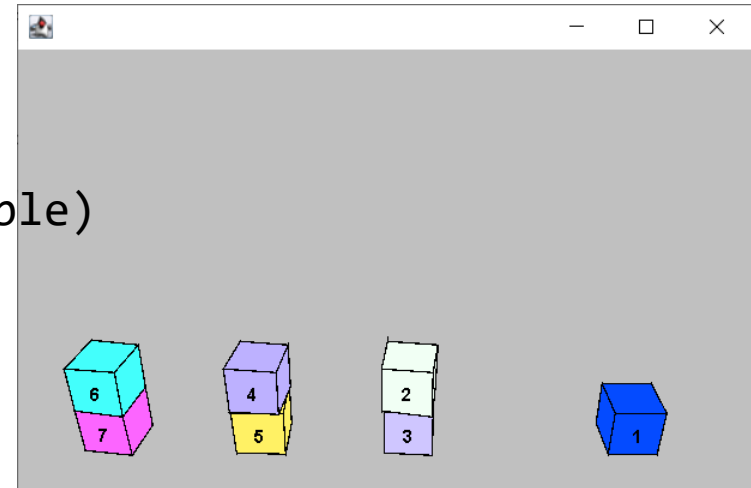
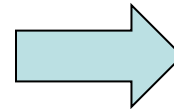


Recap: Blocks World

Environment



`move(b1,table)`



Blocks World properties: **deterministic, static, & single agent**. Therefore, (apart for initial state) agent does not have to look and we can **remove percept channel**

Keep it

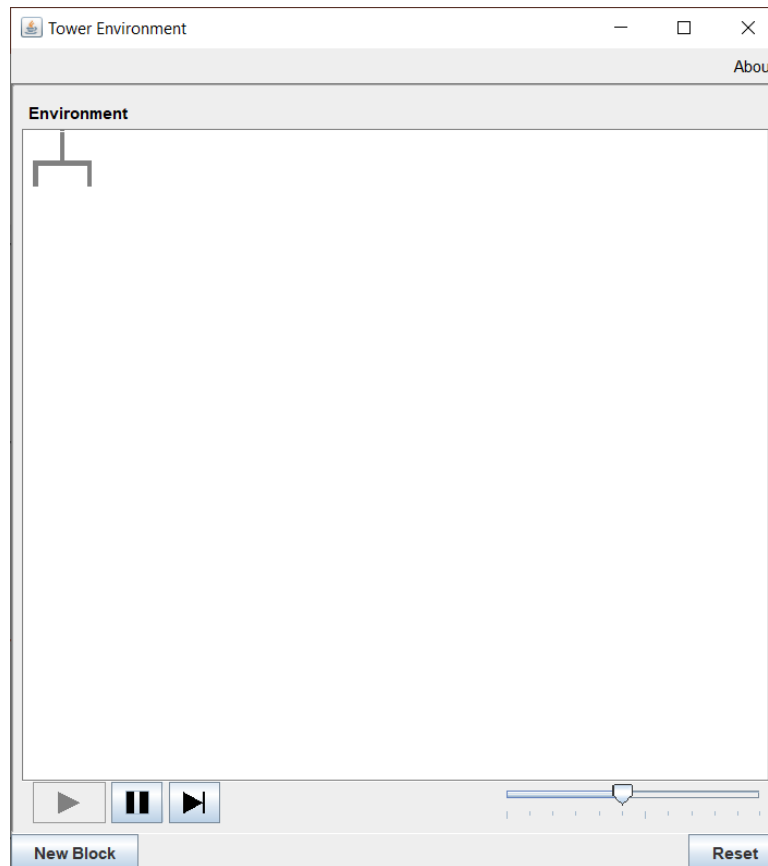
Database

```
on (b1,b2)
on (b2,b3)
on (b3,table)
on (b4,b5)
on (b5,table)
on (b6,b7)
on (b7,table)
```

`delete(b1,b1) +`
`insert(b1,table)`

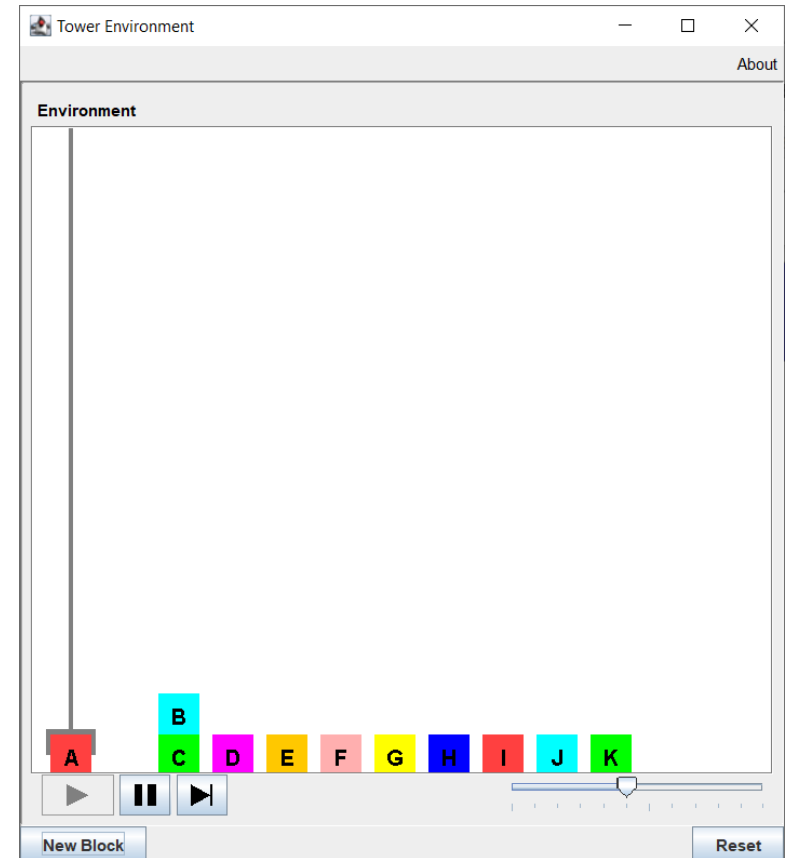
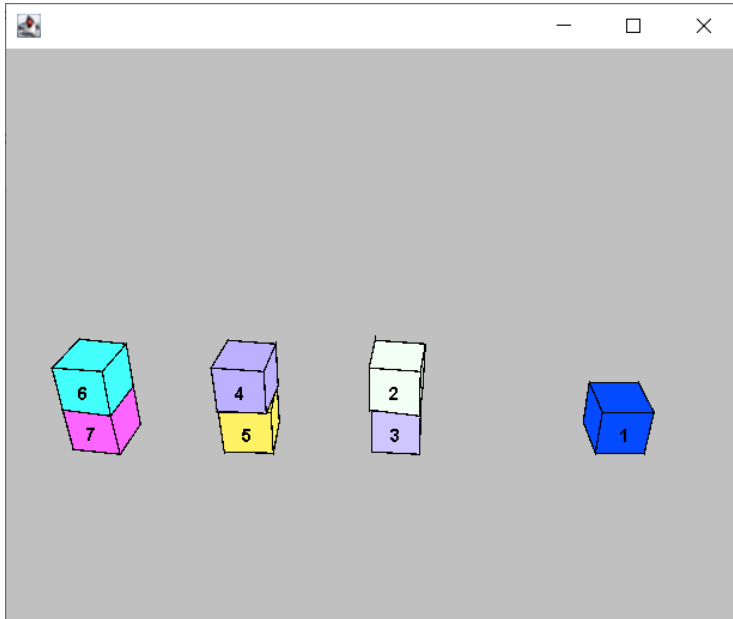
(using built-in
database actions)

```
on (b1,table)
on (b2,b3)
on (b3,table)
on (b4,b5)
on (b5,table)
on (b6,b7)
on (b7,table)
```



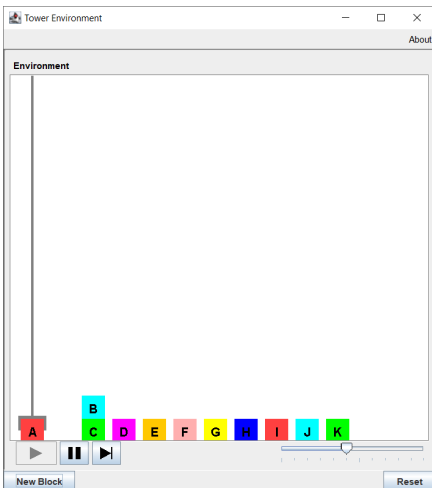
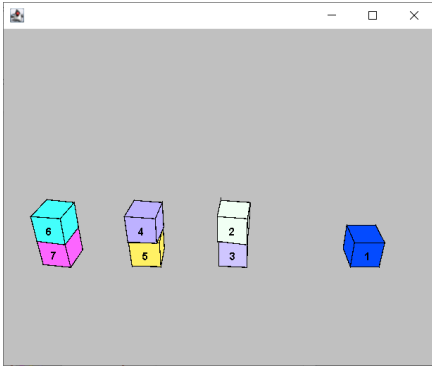
THE TOWER WORLD

Blocks versus Tower World



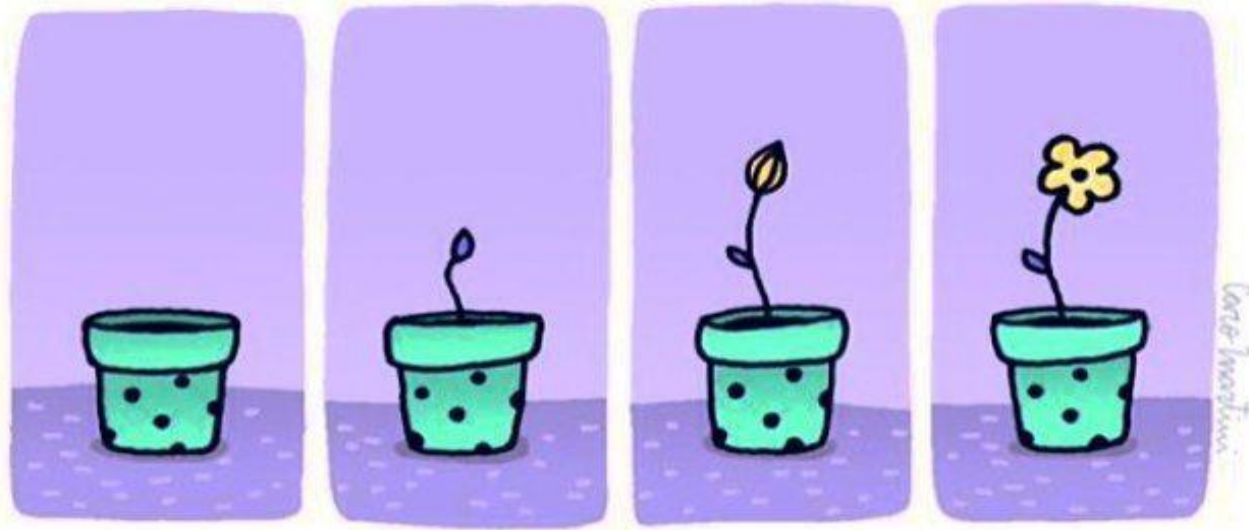
What differences are there between the Blocks World and Tower World?

Blocks versus Tower World



- Blocks must be introduced by user (interacting with interface)
- Gripper that visibly moves and **takes time** to move blocks
- Gripper can **pick up** and **put down** blocks, and move back to its initial state (left top corner)
- User can move blocks too! Interference also by removing blocks from gripper!
- Block names are **numbers** in Blocks World but (capital) **letters** in Tower World

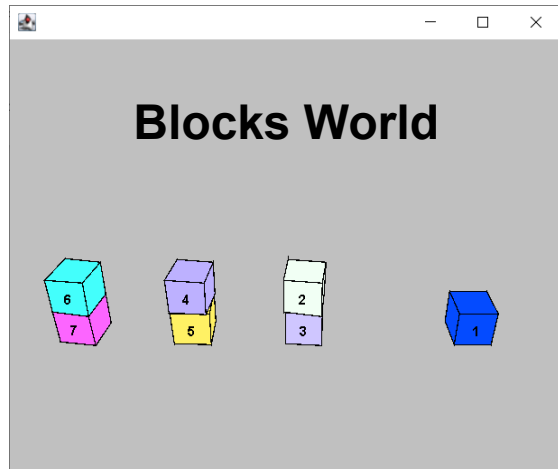
What differences are there between the Blocks World and Tower World?



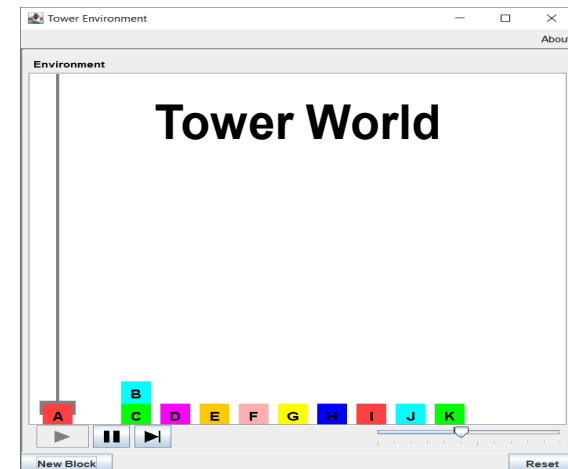
some things take time

DURATIVE ACTIONS

Instantaneous & durative actions

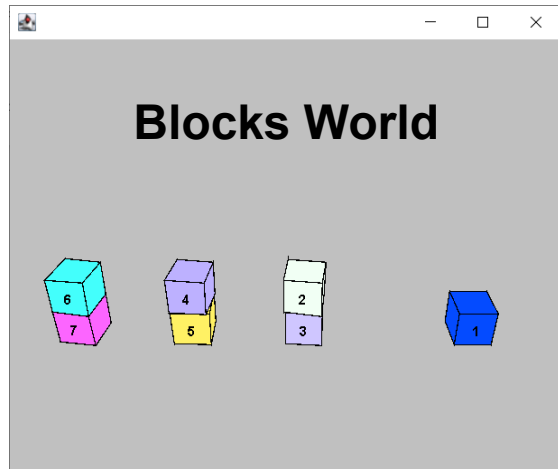


- *Single instantaneous* action `move(X, Y)` that takes (virtually) *no time*.
- Environment-entity needs no time to execute action.



- *Three durative* actions `putdown(X, Y)`, `pickup(X)` and `nil` that take *time to complete*.
- Environment-entity needs time to execute an action.

Instantaneous actions

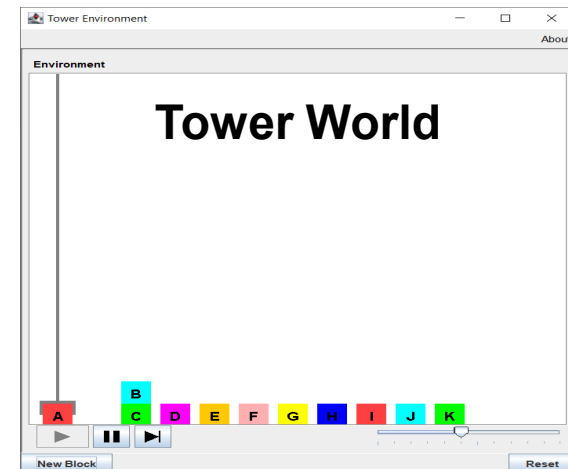


- *Single* **instantaneous** action `move(X, Y)` that takes (virtually) *no time*.
- Environment-entity needs no time to execute action.

- Time needed to execute action **can be neglected**
- Effects of an action are achieved **instantly**.
- Perception of these changes is **immediate**.
- Action always **succeeds (when precondition holds)**.

Durative actions

- Time needed to execute action **can NOT be neglected**
- Effects of an action are achieved **only when the action has been executed completely.**
- Perception of changes due to action **happen after some time, when effects are achieved.**



- *Three **durative** actions* putdown(*X*, *Y*), pickup(*X*) and nil that take *time to complete*.
- Environment-entity needs time to execute an action.

PROBLEM 1: Durative Actions May Fail!

Something may **interfere** & **prevent action success**:

- *Tower World*: User may interfere, e.g., remove block from gripper or move block elsewhere
- *BW4T*: Environment may block action completion, e.g., robot cannot go into a room if it is occupied and gets stuck (multiple robots BW4T)

Key insight here:

→ Agent may need to **monitor** if what it is trying to do is working out, i.e., successful, and, if not **revise** its plan of action.

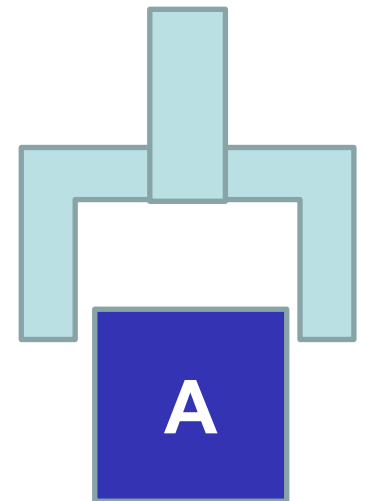
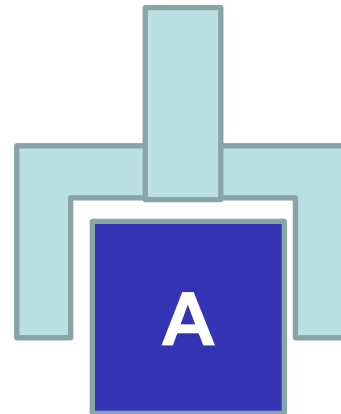
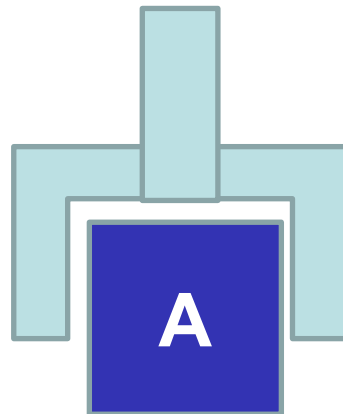
Monitoring Tower World Gripper

not(holding(_))

holding(a)

holding(a)

not(holding(_))



PROBLEM 2: Thinking faster than acting



- While action is taking time to complete, agent can decide on new action to take...
- *BUT what happens if agent decides to take new action while previous one is not completed?*

Performing a Durative Action...

...while entity is already performing a (durative) action:

1. may **be ignored**, e.g., picking up block **b** will be ignored when gripper is already holding another block (Tower World)
→ *Performing the action has no effect.*
2. may **override/cancel** action that is being performed, e.g., going to a place cancels previous action (BW4T)
→ *Overwriting the (same) action cancels that action!*
3. may **be executed in parallel**, e.g., shooting and walking (UT3)

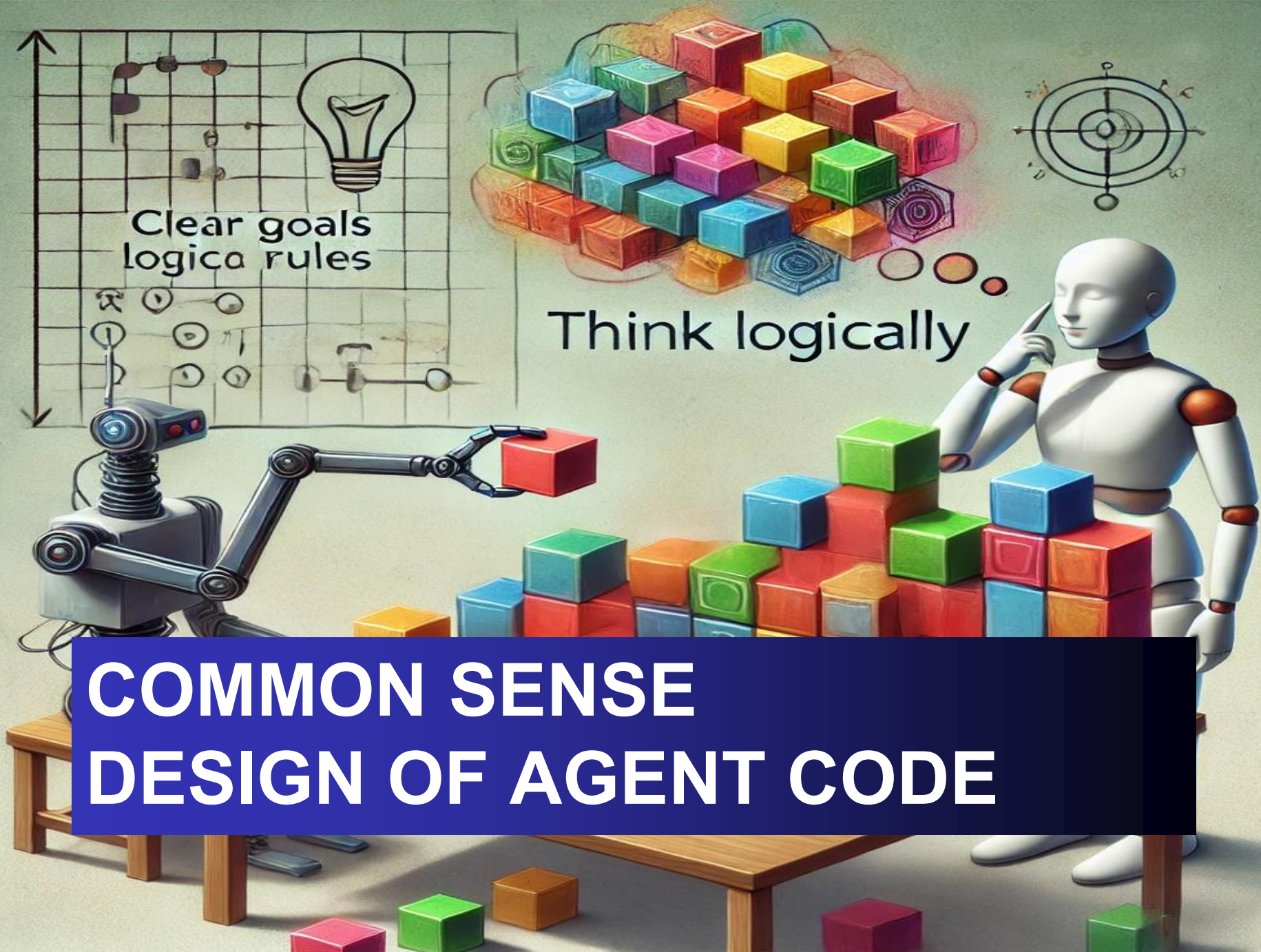
Key insight here:

→ ***Need to keep track of what agent is doing: Only ask environment to execute new action if no action is in progress!***

Recap: Tower World Properties

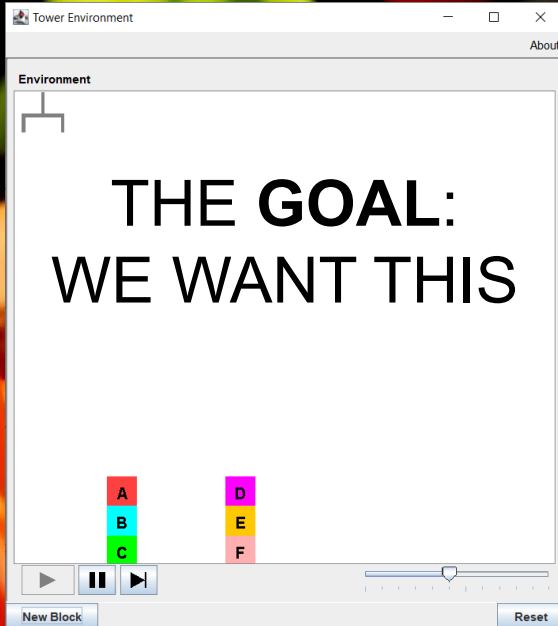
1. **Fully observable**: agent can see everything relevant for decision making.
2. **Deterministic**: action effects are predictable.
3. **Dynamic**: environment can change even if agent does nothing because user can interact via GUI.
4. **Discrete**: finite list of blocks; *NB: continuous gripper movement not relevant for decision making.*
5. **Multi agent**: there is a single gripper entity to connect an agent to, but also a user that can change the environment.

We need channels (percepts) because of 3 (and 5).



COMMON SENSE DESIGN OF AGENT CODE

Start with the main ingredients



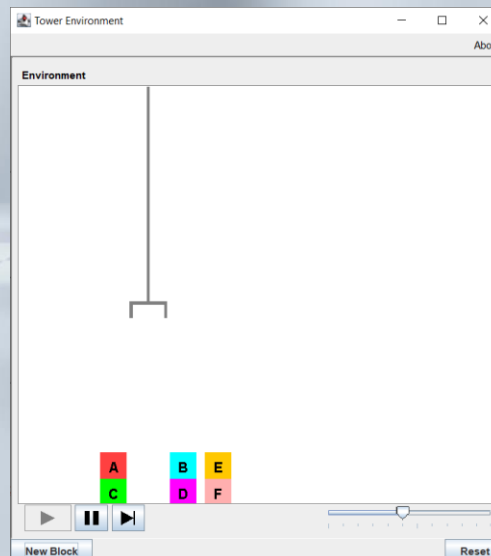
ACTION

BY DOING THIS:

```
pickup(X)  
nil  
putdown(X,Y)
```

And come up with a strategy

A **strategy** or **policy** is a function or a set of rules that defines the agent's behaviour by **mapping states to actions**.

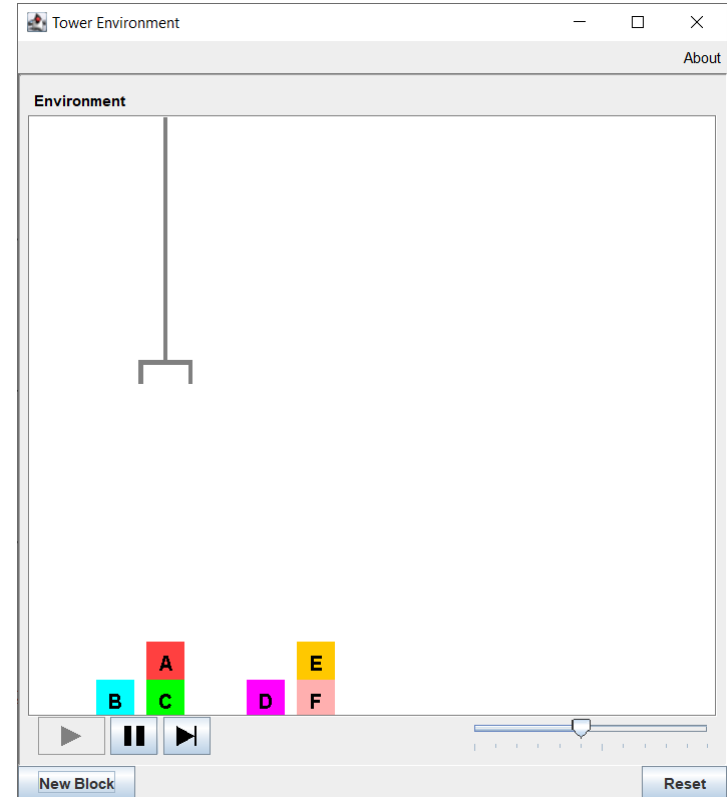


→ pickup(a)

When to perform pickup(X)?

Pick up a block when:

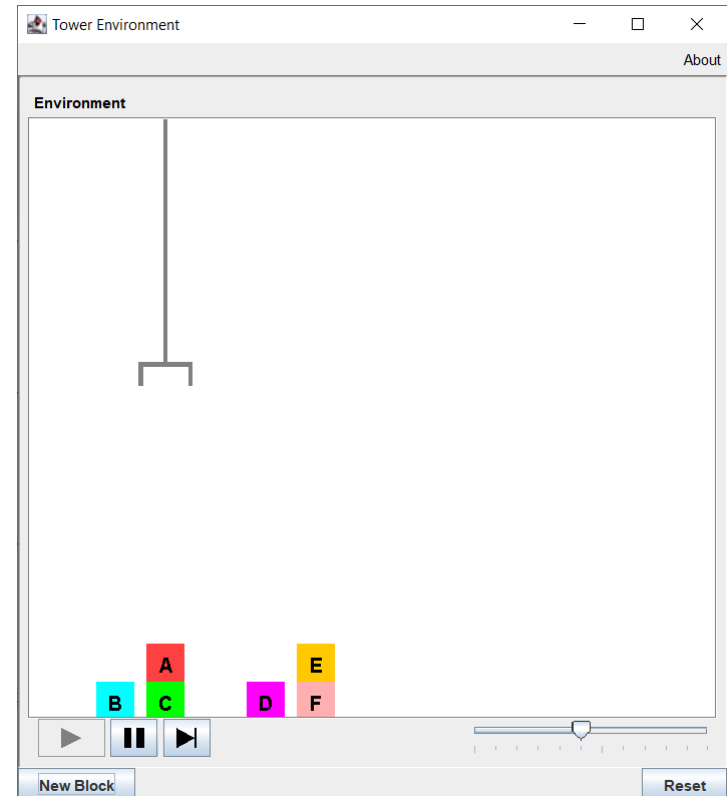
- ...



When to perform pickup(**X**)?

Pick up a block when:

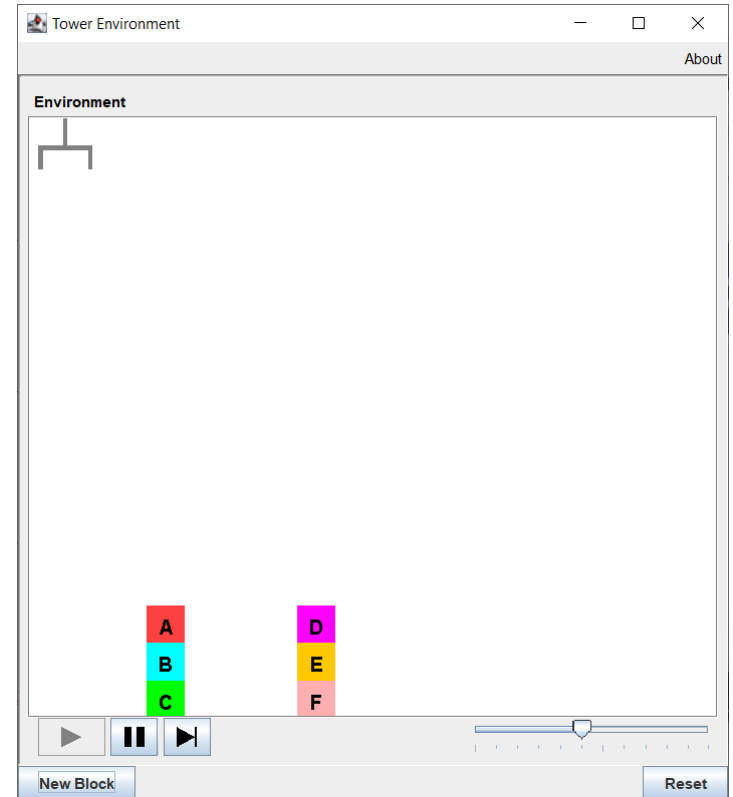
- It is sitting on top of a block it should not sit on, i.e. it is **misplaced**
- It can be used to **constructively** get closer to the goal, i.e. constructive means we do not need to 'undo' the move ever again



When to perform nil?

Move the gripper to the corner when:

- ...

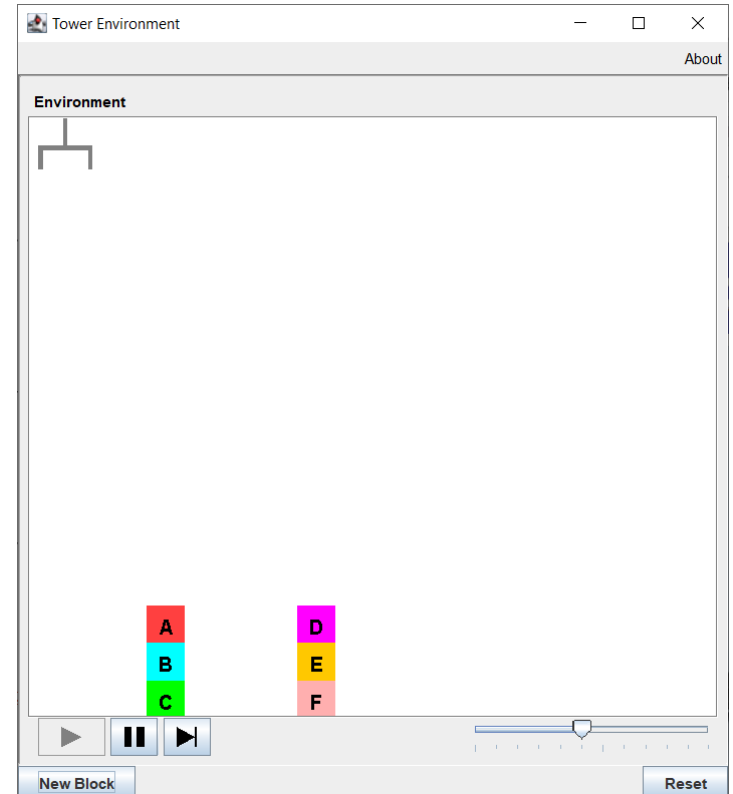


When to perform nil?

Move the gripper to the corner when:

- The goal has been achieved, i.e. when the towers have been built

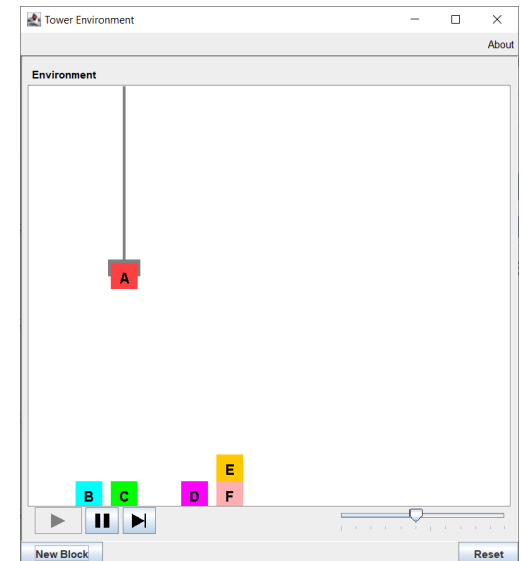
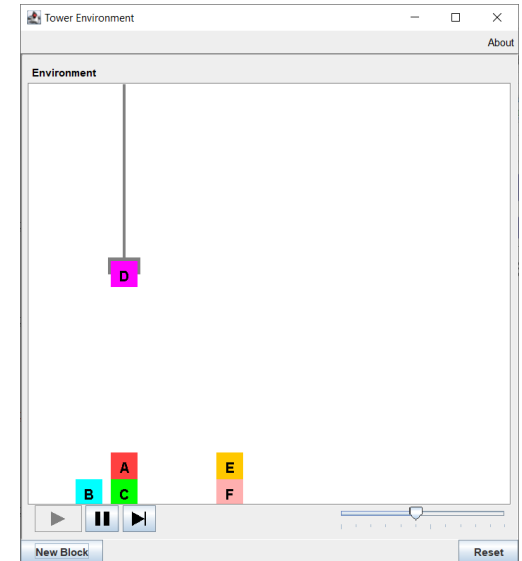
In other words, *perform nil when there is **nothing more to do***



When to perform putdown(*X*, *Y*)?

Put down a block and move it to *Y*, when:

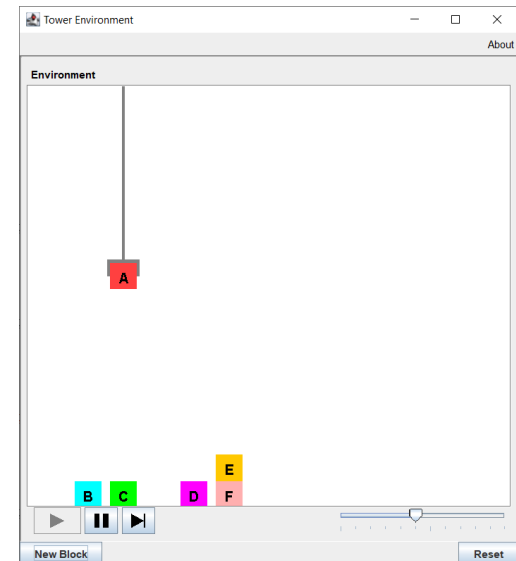
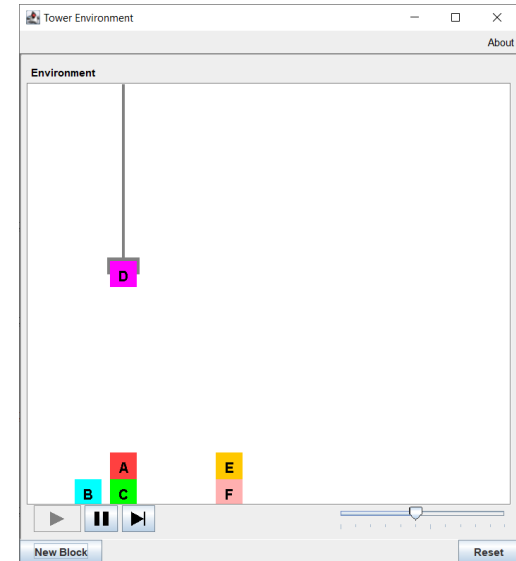
- ...



When to perform putdown(*X*, *Y*)?

Put down a block *when you're holding it* and move it to *Y*, when:

- Move to (*Y*=)table if it cannot be used constructively right now to make progress towards the goal
- Move on top of block (*Y*) if it can be used constructively to make progress towards the goal



Action Rule Summary

- if a block is misplaced, pick it up
- if a block can be used constructively, pick it up
- if there is nothing more to do, do nil
- if a block cannot be used constructively, when holding that block, move it to the table
- if a block can be used constructively by putting it down on another block, when holding that block, move it onto the other block

Which Rule Order?

- if a block is misplaced, pick it up
- if a block can be used constructively, pick it up
- if there is nothing more to do, do nil
- if a block cannot be used constructively, when holding that block, move it to the table
- if a block can be used constructively by putting it down on another block, when holding that block, move it onto the other block

Action Priorities → Rule Order

One way to think about this is to ask **which action should get highest priority:**

1. ...
2. ...
3. ...

ACTION

BY DOING THIS:

```
pickup(X)  
nil  
putdown(X,Y)
```

Action Priorities → Rule Order

One way to think about this is to ask **which action should get highest priority**:

1. `putdown(X, Y)`, because when holding a block, that block must go down again
2. `pickup(X)`, because there is still some work to do
3. `nil`, because there is nothing else to do

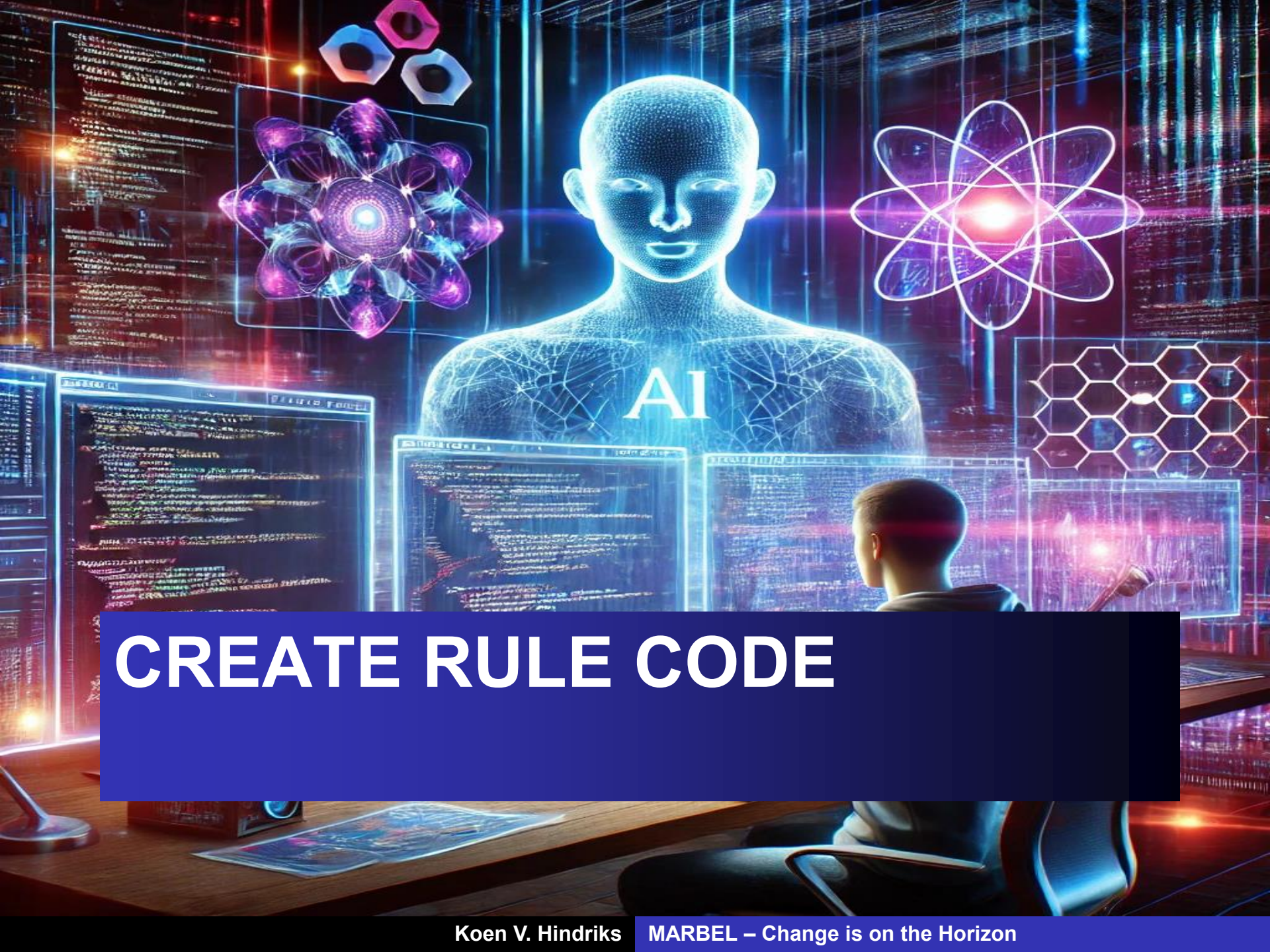
Reorder Rule Priorities

-  if a block cannot be used constructively, when holding that block, move it to the table
-  if a block can be used constructively by putting it down on another block, when holding that block, move it onto the other block
-  if a block is misplaced, pick it up
-  if a block can be used constructively, pick it up
-  if there is nothing more to do, do nil

Prioritize Progress Towards Goal

-  if a block can be used constructively by putting it down on another block, when holding that block, move it onto the other block
-  if a block cannot be used constructively, when holding that block, move it to the table
-  if a block can be used constructively, pick it up
-  if a block is misplaced, pick it up
- if there is nothing more to do, do nil





CREATE RULE CODE

Re-Using Key BW Concepts

- Query for when a constructive move can be made, i.e. move block X onto Y:

`constructiveMove(X, Y)`

- Query for when a block X is misplaced:

`misplaced(X)`

MARBEL Code for first rule

- if a block can be used constructively by putting it down on another block, when holding that block, move it onto the other block

```
if constructiveMove(X, Y), holding(X) then putdown(X, Y).
```

- if a block cannot be used constructively, when holding that block, move it to the table
- if a block can be used constructively, pick it up
- if a block is misplaced, pick it up
- if there is nothing more to do, do nil

MARBEL Code for second rule

```
if constructiveMove(X, Y), holding(X) then putdown(X, Y).
```

- if a block cannot be used constructively, when holding that block, move it to the table

```
if holding(X) then putdown(X, table).
```

- if a block can be used constructively, pick it up
 - if a block is misplaced, pick it up
 - if there is nothing more to do, do nil
- EXPLOITING RULE ORDER:**
Second rule will only be applicable if previous rule fails!
In other words:
We do not need to write code for 'block cannot be used constructively'!

MARBEL Code for third rule

```
if constructiveMove(X, Y), holding(X) then putdown(X, Y).  
if holding(X) then putdown(X, table).
```

- if a block can be used constructively, pick it up

```
if constructiveMove(X, _) then pickup(X).
```

- if a block is misplaced, pick it up
- if there is nothing more to do, do nil

MARBEL Code for fourth rule

```
if constructiveMove(X, Y), holding(X) then putdown(X, Y).  
if holding(X) then putdown(X, table).
```

```
if constructiveMove(X, _) then pickup(X).
```

- if a block is misplaced, pick it up

```
if misplaced(X) then pickup(X).
```

- if there is nothing more to do, do nil

MARBEL Code for fifth rule

```
if constructiveMove(X, Y), holding(X) then putdown(X, Y).  
if holding(X) then putdown(X, table).  
  
if constructiveMove(X, _) then pickup(X).  
  
if misplaced(X) then pickup(X).
```

- if there is nothing more to do, do nil

```
if true then nil.
```

EXPLOITING RULE ORDER again:

If all of the above rules fail, there is nothing more to do!

In other words:
We can simply use the Prolog query 'true' as rule condition!

Re-organizing and cleaning up

```
if holding(X) then {  
    if constructiveMove(X, Y) then putdown(X, Y).  
    if true then putdown(X, table).  
}  
  
if constructiveMove(X, _) then pickup(X).  
  
if misplaced(X) then pickup(X).  
  
if true then nil.
```

EXPLOITING SHARED CONDITIONS (QUERY PARTS):
Using NESTING of rules construct { ... }

Strategy we can use as body of our decision module

```
if holding(X) then {  
    if constructiveMove(X, Y) then putdown(X, Y).  
    if true then putdown(X, table).  
}  
  
if constructiveMove(X, _) then pickup(X).  
  
if misplaced(X) then pickup(X).  
  
if true then nil.
```

WE SIMPLY TRANSLATED COMMON SENSE INTO CODE!!



CREATING A MAS FOR THE TOWER WORLD

Create a new mas project

Creates mas2g file with empty launch policy.

We want to use Tower World environment:

- collect from <https://github.com/eishub/tower>
- download latest release
- copy into project

Add **environment use clause**

In towerWorld.mas2g file, at the top:

use "tower-1.4.1.jar".

The Tower World environment has no parameters (no 'with' part to initialize TW).

Add an **agent definition**

We need:

- An **agent name**: towerBuilderAgt
- **Module names** + types:
 - towerBuilder: Module to put our **strategy** (rules) in; a strategy goes into a module for *decisions*
 - towerInit: Module for **setting the agent's goal**; initial goals are set in a module for *init*(ialisation)
- **Percepts** + handlers: (*check env manual*)
 - on/2 for updating block positions with **replace**
 - holding/1 for updating gripper status w/ **replace**

Add an agent definition

```
use "tower-1.4.1.jar".
```

```
define towerBuilderAgt {  
  use towerBuilder for decisions.  
  use towerInit for init.
```

We'll put our
rule-based
strategy here

```
  replace on/2.  
  replace holding/1.
```

Here used for
goal/agenda
setting

```
}
```

Percept
processing

```
launchpolicy {  
  when * launch towerBuilderAgt.
```

```
}
```

Launch agent when
entity becomes
available

Create module towerBuilder

```
use towerKnowledge.
```

Knowledge we will use
(Prolog) to define rule
queries/conditions

```
module towerBuilder {
```

```
  if holding(X) then {  
    if constructiveMove(X, Y) then putdown(X, Y).  
    if true then putdown(X, table).  
  }
```

```
  if constructiveMove(X, _) then pickup(X).
```

```
  if misplaced(X) then pickup(X).
```

```
  if true then nil.
```

Our rule-based
strategy

```
}
```


Create Prolog file **towerKnowledge**

- Declare predicates that are used but not defined dynamically:

```
:- dynamic holding/1, on/2.
```

- Introduce predicate definitions that are used:

```
constructiveMove(X, Y) :- ... .
```

```
misplaced(X) :- ... .
```

Create module **towerInit**

```
use towerKnowledge.
```

Tell MARBEL
interpreter that Prolog
will be used

```
module towerInit {
```

```
  if true then insert(agenda([[a,b,c], [d,e,f]])).
```

```
}
```

Define agent's **agenda**: a list of the
towers that we want.

We represent towers also as lists, so
our agenda is a *list of lists*.

This predicate is
used but...

...not defined →
declare in Prolog
file!

Keep defining...!

- Everything seems to be in place, except for those Prolog definitions...
- Let's re-use our insights from last lecture:
 - copy dfs for `block/1`, `clear/1`, and `tower/1`
 - define agenda-based versions of `tower` and `on`
 - definitions for constructive and misplaced blocks

Copy BW Prolog definitions

```
block(X) :- on(X, _); holding(X).
```

```
clear(X) :- block(X), not( on(_, X) ).  
clear(table).
```

```
tower([X]) :- on(X, table).  
tower([X, Y | T]) :- on(X, Y), tower([Y | T]).
```

Any problems here that we need to fix for the Tower World?

Agenda-based versions

- We use: `agenda([[a,b,c], [d,e,f]])`
- But we need a single (list) goal tower...
- Let's introduce a ***helper*** predicate to fix that:

```
goal(Goal) :-  
    agenda(Agenda),  
    member(Goal, Agenda).
```

Agenda-based versions

```
agenda_tower([X]) :- agenda_on(X, table).  
agenda_tower([X,Y|T]) :-  
    agenda_on(X, Y), agenda_tower([Y|T]).
```

```
agenda_on(X, Y) :-  
    goal(Goal), append(_, [X, Y|_], Goal).
```

```
goal(Goal) :- agenda(Agenda), member(Goal, Agenda).
```

```
constructiveMove(X, Y) :- agenda_tower([X,Y|T]),  
    tower([Y|T]), clear(X), clear(Y).  
misplaced(X) :- agenda_tower([X|T]),  
    not(tower([X|T])), clear(X), not(on(X,table)).
```

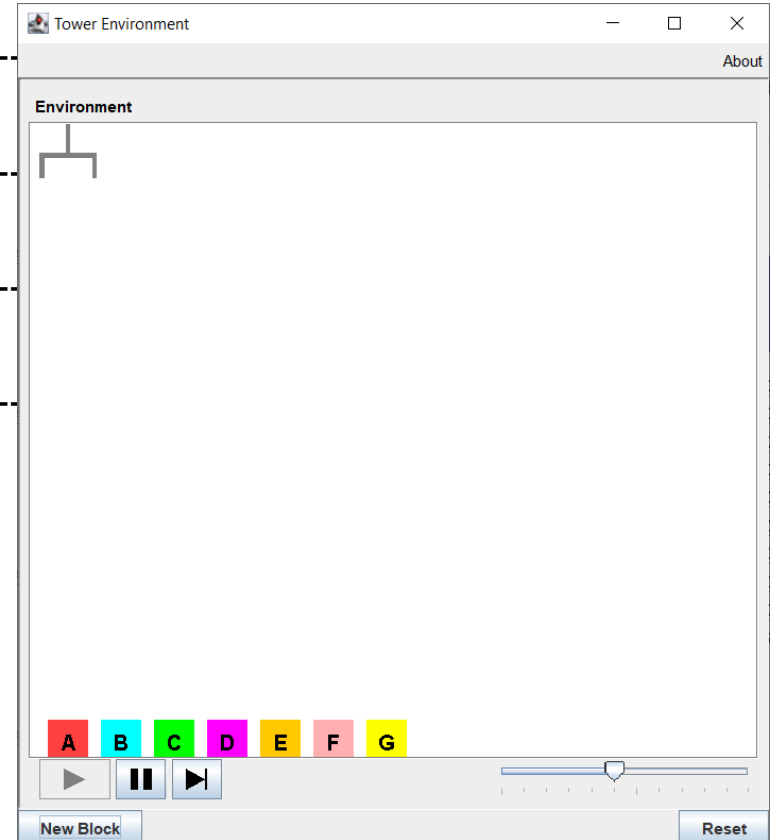
Finally... no compile errors!



Let's run the agent and see what happens...

Nothing happens... only nil

```
[towerBuilderAgt] ++++++ Cycle 61981 ++++++  
[towerBuilderAgt] performed 'nil'.  
[towerBuilderAgt] ++++++ Cycle 61982 ++  
[towerBuilderAgt] performed 'nil'.  
[towerBuilderAgt] ++++++ Cycle 61983 ++  
[towerBuilderAgt] performed 'nil'.  
[towerBuilderAgt] ++++++ Cycle 61984 ++  
[towerBuilderAgt] performed 'nil'.  
[towerBuilderAgt] ++++++ Cycle 61985 ++  
User(event) paused me in cycle 61985.
```



What is wrong?

→ Clearly, there is no constructive move...

Debugging

Check the (parts of the) definition of a constructive move in the *Interactive Console*

→agenda_tower does not work...

→agenda_on does not work... There are no blocks on the table...!

Simple fix: add...

```
agenda_on(X, table) :-
```

```
    goal(Goal), append(_, [X], Goal).
```

Let's check our fix



Let's run the agent and see what happens...



When all things are done

ORGANIZATION

Read MARBEL Programming Guide [Chapter 4](#).